

Lecture 2: Introduction

CS3212 Algorithms

Aya Zirikly
08/27/2026

Big O

$$f(n) = O(g(n))$$

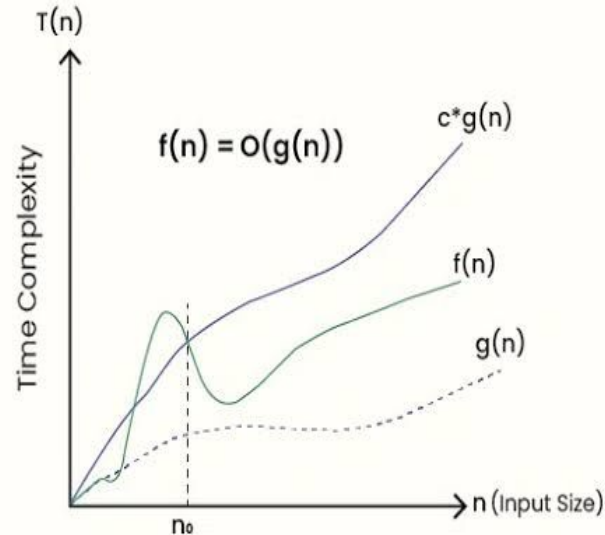
It formally means there exist two positive constants, c and n_0 , such that:

$$f(n) \leq c * g(n) \text{ for all } n \geq n_0$$

- **c (The Multiplier):** Scales the upper bound function to account for hardware or constant factors.
- **n_0 (The Starting Line):** Specifies the exact input size from which the inequality must hold true forever.

Big-O (what is n_0)

n_0 represents the threshold input size beyond which the upper bound of your function strictly holds true. It marks the starting point where the "long-term" or asymptotic behavior of the algorithm begins, allowing you to safely ignore smaller inputs where minor terms might fluctuate



Big O Analysis

$|f(n)|$ is asymptotically bounded above by $g(n)$ up to constant factor c



Why do we need n^0

Consider two algorithms for sorting an array:

- **Algorithm A (Linear Growth):** Time = $10 * n$
- **Algorithm B (Quadratic Growth):** Time = $2 * (n^2)$

Let's test them on different input sizes:

- **At $n = 2$:** Algorithm A takes $10 * 2 = 20$ operations. Algorithm B takes $2 * (2^2) = 8$ operations. *Here, Algorithm B appears faster!*

Why do we need n^0

Consider two algorithms for sorting an array:

- **Algorithm A (Linear Growth):** Time = $10 * n$
- **Algorithm B (Quadratic Growth):** Time = $2 * (n^2)$

Let's test them on different input sizes:

- **At $n = 2$:** Algorithm A takes $10 * 2 = 20$ operations. Algorithm B takes $2 * (2^2) = 8$ operations. *Here, Algorithm B appears faster!*
- **At $n = 5$:** Algorithm A takes $10 * 5 = 50$ operations. Algorithm B takes $2 * (5^2) = 50$ operations. *They are identical.*

Why do we need n^0

Consider two algorithms for sorting an array:

- **Algorithm A (Linear Growth):** Time = $10 * n$
- **Algorithm B (Quadratic Growth):** Time = $2 * (n^2)$

Let's test them on different input sizes:

- **At $n = 2$:** Algorithm A takes $10 * 2 = 20$ operations. Algorithm B takes $2 * (2^2) = 8$ operations. *Here, Algorithm B appears faster!*
- **At $n = 5$:** Algorithm A takes $10 * 5 = 50$ operations. Algorithm B takes $2 * (5^2) = 50$ operations. *They are identical.*
- **At $n \geq 6$:** For $n = 10$: Algorithm A takes 100 ops, Algorithm B takes 200 ops. For $n = 100$: Algorithm A takes 1,000 ops, Algorithm B takes 20,000 ops.

Why do we need n_0

Consider two algorithms for sorting an array:

- **Algorithm A (Linear Growth):** Time = $10 * n$
- **Algorithm B (Quadratic Growth):** Time = $2 * (n^2)$

Let's test them on different input sizes:

- **At $n = 2$:** Algorithm A takes $10 * 2 = 20$ operations. Algorithm B takes $2 * (2^2) = 8$ operations. *Here, Algorithm B appears faster!*
- **At $n = 5$ (The Crossover Point):** Algorithm A takes $10 * 5 = 50$ operations. Algorithm B takes $2 * (5^2) = 50$ operations. *They are identical.*
- **At $n \geq 6$ (Beyond n_0):** For $n = 10$: Algorithm A takes 100 ops, Algorithm B takes 200 ops. For $n = 100$: Algorithm A takes 1,000 ops, Algorithm B takes 20,000 ops.

Big-O Step-by-Step Proof Example 3 (Different n_0)

Claim: Prove that $f(n) = n^2 + 100n = O(n^2)$.

Method A (Scale constant):

- For $n \geq 1$, $100n \leq 100n^2 \Rightarrow n^2 + 100n \leq 101n^2$ ($c = 101$, $n_0 = 1$).

Method B (Scale threshold):

- Assume $n \geq 100$. Then $100n \leq n * n = n^2$.
- Substitute: $n^2 + 100n \leq n^2 + n^2 = 2n^2$.
- Yields $c = 2$ and $n_0 = 100$.

Key Takeaway: (c , n_0) are **not unique**. Any valid pair proves asymptotic membership.

More (c,n0)

1. Assume $n \geq 10$.
2. Multiply both sides by 10 to express $100n$ in terms of n^2 : $100n \leq 10 * n^2$
3. Substitute back into $f(n)$: $f(n) = n^2 + 100n \leq n^2 + 10n^2 = 11n^2$

Yields: $c = 11$ and $n_0 = 10$.

- **Verification at $n = 10$:**
 - $f(10) = (10)^2 + 100(10) = 100 + 1000 = 1100$
 - $c * g(10) = 11 * (10)^2 = 11 * 100 = 1100$
 - Holds true! ($1100 \leq 1100$)

More (c,n0)

Get the constant **c** as close to 1 as possible ($c = 1.1$) by setting a higher threshold.

1. Assume **$n \geq 1000$** .
2. Divide 100 by 1000 to express $100n$ in terms of n^2 : $100n \leq 0.1 * n^2$
3. Substitute back into $f(n)$: $f(n) = n^2 + 100n \leq n^2 + 0.1n^2 = \mathbf{1.1n^2}$

Yields: $c = 1.1$ and $n_0 = 1000$.

- **Verification at $n = 1000$:**
 - $f(1000) = (1000)^2 + 100(1000) = 1,000,000 + 100,000 = \mathbf{1,100,000}$
 - $c * g(1000) = 1.1 * (1000)^2 = 1.1 * 1,000,000 = \mathbf{1,100,000}$
 - Holds true! ($1,100,000 \leq 1,100,000$)

Asymptotic Lower Bound — Big-Omega(Ω)

Big-Omega (Ω): The **Floor** (Best-Case / Lower Bound).

"The runtime will not be better than this."

Asymptotic Lower Bound — Big-Omega (Ω)

- **Formal Definition:** $f(n) = \Omega(g(n))$ if there exist positive constants $c > 0$ and $n_0 \geq 0$ such that:

$$0 \leq c * g(n) \leq f(n) \text{ for all } n \geq n_0$$

- **Intuition:** $g(n)$ serves as an **asymptotic lower bound** (floor) on $f(n)$. $f(n)$ grows at least as fast as $g(n)$.
- **Usage in Algorithms:** Used to state fundamental limitations (e.g., "Any comparison sort requires $\Omega(n \log n)$ operations").

Big-Omega (Ω) Step-by-Step Proof Example

- **Claim:** Prove that $f(n) = (1/2)n^2 - 5n = \Omega(n^2)$.
- **Goal:** Find $c > 0$, $n_0 \geq 0$ such that $(1/2)n^2 - 5n \geq c * n^2$.

Step-by-Step Proof:

1. We want to ensure that subtracting $5n$ does not reduce $(1/2)n^2$ below $c * n^2$.
2. Set up condition: Require $5n \leq (1/4)n^2$.
3. Divide both sides by n (assuming $n > 0$): $5 \leq (1/4)n \Rightarrow n \geq 20$.
4. For $n \geq 20$:
$$f(n) = (1/2)n^2 - 5n \geq (1/2)n^2 - (1/4)n^2 = (1/4)n^2$$
5. Set $c = 1/4$ and $n_0 = 20$.

Conclusion: $f(n) = (1/2)n^2 - 5n = \Omega(n^2)$ [Q.E.D.]

Is $1/4$ the ONLY Choice?

Is $1/4$ the ONLY Choice? (No!)

Is $1/4$ the ONLY Choice? (No!)

- Require $5n \leq (4/10)n^2$
- Divide by n : $5 \leq (4/10)n \Rightarrow n \geq 12.5 \Rightarrow$
 $n_0 = 13$
- $f(n) \geq (1/2)n^2 - (4/10)n^2 =$
 $(1/10)n^2$
- **Result: $c = 1/10$, $n_0 = 13$** (*Lower threshold, but smaller constant c*)

- Require $5n \leq (1/10)n^2$
- Divide by n : $5 \leq (1/10)n \Rightarrow n \geq 50 \Rightarrow$ **$n_0 = 50$**
- $f(n) \geq (1/2)n^2 - (1/10)n^2 =$
 $(4/10)n^2$
- **Result: $c = 2/5$, $n_0 = 50$** (*Larger constant c , but requires waiting until $n = 50$*)

Rule of Thumb

you see $a * n^2 - b * n$ in a Big-Omega proof:

- Set $b * n \leq (a / 2) * n^2$ (half of the leading coefficient).

Key Properties of the Constant c

- ❑ **Must be strictly positive:** $c > 0$
- ❑ **Must be fixed and independent of n :** c cannot be a variable function like $c = n$ or $c = \log n$
- ❑ **Not unique:** Multiple valid values for c can prove the same asymptotic claim, typically trading off against the choice of threshold n_0

Big-Omega (Ω) Step-by-Step Proof Example

Claim: Prove that $f(n) = 3n^2 - 12n = \Omega(n^2)$.

Goal: Find $c > 0$, $n_0 \geq 0$ such that $3n^2 - 12n \geq c * n^2$.

Step-by-Step Proof: We want to ensure that subtracting $12n$ does not reduce $3n^2$ below $c * n^2$. Strategy: Sacrifice half of the main term ($3n^2$) to absorb the subtracted term ($12n$). Half of $3n^2$ is $(3/2)n^2$.

Set up condition: Require $12n \leq (3/2)n^2$. Divide both sides by n (assuming $n > 0$): $12 \leq (3/2)n \Rightarrow n \geq 24/3 \Rightarrow n \geq 8$.

For $n \geq 8$: $f(n) = 3n^2 - 12n \geq 3n^2 - (3/2)n^2 = (3/2)n^2$

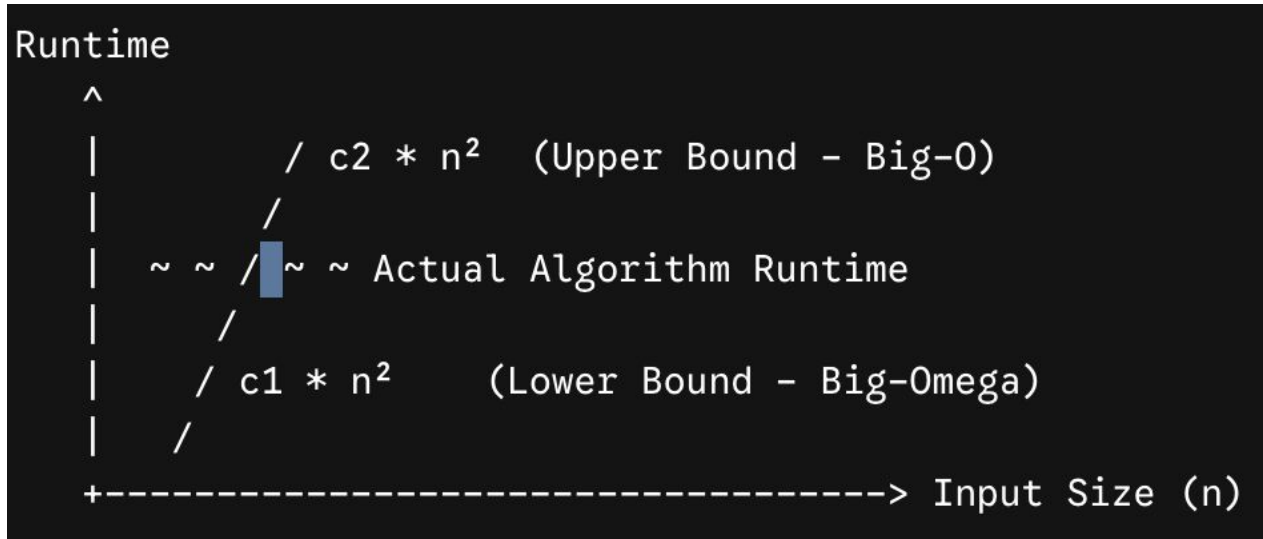
Set $c = 3/2$ and $n_0 = 8$.

Conclusion: $f(n) = 3n^2 - 12n = \Omega(n^2)$ [Q.E.D.]

Asymptotic Tight Bound — Big-Theta(Θ)

An algorithm's performance is tightly bounded from both above and below.

Big-Theta (Θ): *"The runtime grows at precisely this rate."*

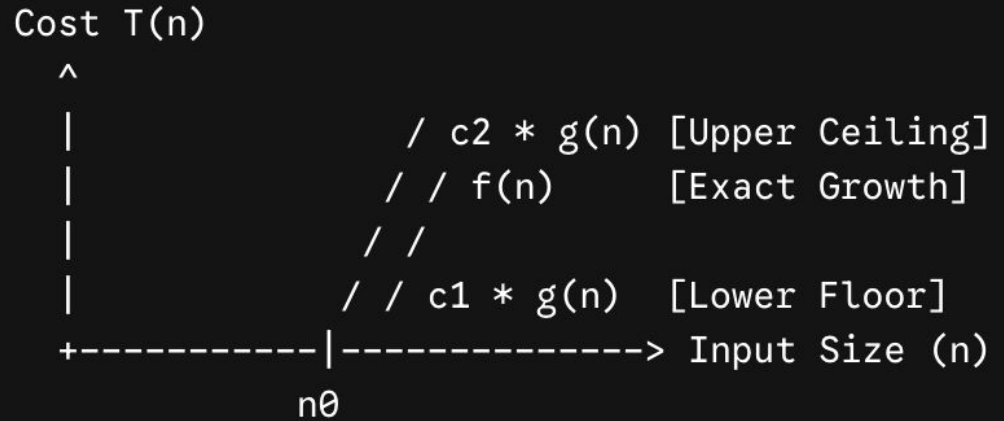


Asymptotic Tight Bound — Big-Theta(Θ)

Formal Definition: $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Combined Condition: There exist positive constants $c_1, c_2 > 0$ and $n_0 \geq 0$ such that:

$$0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n) \text{ for all } n \geq n_0$$



Big-Theta Step-by-Step Proof Example

Claim: Prove that $f(n) = 6n^2 + 2n = \Theta(n^2)$.

Proving $f(n) = 6n^2 + 2n =$
 $\Theta(n^2)$

Look at $f(n) = 6n^2 + 2n$. When n grows very large (e.g., $n = 1,000,000$), which term dominates the growth?

$6n^2$ clearly dominates! As $n \rightarrow \infty$, the $+2n$ becomes negligible compared to $6n^2$.
Therefore, $f(n)$ behaves like n^2 . We will formally prove that $f(n)$ is tightly bound by n^2 using Big-Theta definition!

The Big-Theta Definition

To prove **$f(n) = \Theta(g(n))$** , we must find 3 positive constants: **c_1 , c_2 , and n_0** such that:

$$c_1 * g(n) \leq f(n) \leq c_2 * g(n) \text{ for all } n \geq n_0$$

Upper Bound (Big-O)

$$f(n) \leq c_2 * n^2$$

Shows $f(n)$ does not grow faster than $c_2 * n^2$
for large n .

Lower Bound (Big-Omega)

$$c_1 * n^2 \leq f(n)$$

Shows $f(n)$ does not grow slower than $c_1 * n^2$
for large n .

Two-Step

Find Upper Bound (c_2 and n_{0_1}):

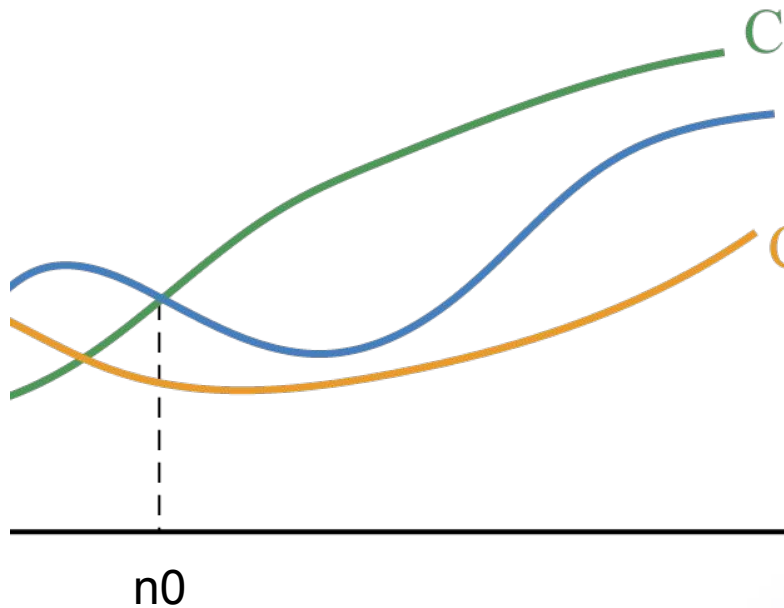
Show $6n^2 + 2n \leq c_2 * n^2$

Find Lower Bound (c_1 and n_{0_2}):

Show $c_1 * n^2 \leq 6n^2 + 2n$

Combine Constants:

Choose $n_0 = \max(n_{0_1}, n_{0_2})$ so both bounds hold together!



✓ Step 1: Big-O Proof

Upper Bound Derivation

For all $n \geq 1$, we know that $n \leq n^2$, so $2n \leq 2n^2$.

$$f(n) = 6n^2 + 2n \leq 6n^2 + 2n^2 = 8n^2$$

Therefore: $6n^2 + 2n \leq 8 * n^2$ for all $n \geq 1$.

🚩 RESULT FOR UPPER BOUND

We have found our upper constants: $c_2 = 8$ and $n_0 = 1$.

This proves $f(n) = O(n^2)$!

? Lower Bound

CHALLENGE

Now we want to show: $c_1 * n^2 \leq 6n^2 + 2n$ for all $n \geq 1$.

✓ Step 2 Solution: Big-Omega

Lower Bound Derivation

Since $n \geq 1$, the term $2n > 0$.

$$6n^2 \leq 6n^2 + 2n \text{ for all } n \geq 1$$

Therefore: $6 * n^2 \leq 6n^2 + 2n$ for all $n \geq 1$.

🚩 RESULT FOR LOWER BOUND

We have found our lower constants: $c1 = 6$ and $n0_2 = 1$.

This proves $f(n) = \Omega(n^2)$!

Combining the Constants

Let's put both bounds together into the master Big-Theta inequality:

Chosen Constants:

$$c_1 = 6$$

$$c_2 = 8$$

$$n_0 = \max(1, 1) = 1$$

$$6 * n^2 \leq 6n^2 + 2n \leq 8 * n^2 \text{ for all } n \geq 1$$

✓ Q.E.D. Claim Proven! $f(n) = \Theta(n^2)$

Classroom Check: Plug In N

Let's test our inequality $6n^2 \leq 6n^2 + 2n \leq 8n^2$ with real values!

n	Lower Bound ($6n^2$)	Actual $f(n) = 6n^2 + 2n$	Upper Bound ($8n^2$)	Is $6n^2 \leq f(n) \leq 8n^2$?
1				
2				
5				
10				

Classroom Check: Plug In N

Let's test our inequality $6n^2 \leq 6n^2 + 2n \leq 8n^2$ with real values!

n	Lower Bound ($6n^2$)	Actual $f(n) = 6n^2 + 2n$	Upper Bound ($8n^2$)	Is $6n^2 \leq f(n) \leq 8n^2$?
1	6	8	8	TRUE ($6 \leq 8 \leq 8$)
2	24	28	32	TRUE ($24 \leq 28 \leq 32$)
5	150	160	200	TRUE ($150 \leq 160 \leq 200$)
10	600	620	800	TRUE ($600 \leq 620 \leq 800$)

Discussion: Choosing Constants



Are $c_1 = 6$, $c_2 = 8$, $n_0 = 1$ the ONLY valid choice of constants?

✓ NO! CONSTANTS ARE NOT UNIQUE!

In Big-Theta proofs, infinitely many valid combinations of (c_1, c_2, n_0) exist.

- **Alternative Choice A:** $c_1 = 1$, $c_2 = 10$, $n_0 = 1$
- **Alternative Choice B:** $c_1 = 6$, $c_2 = 7$, $n_0 = 2$ (since $2n \leq n^2$ for $n \geq 2$, so $6n^2 + 2n \leq 7n^2$)

Rule of thumb: Choose whichever valid constants are easiest to prove!

★ Key Takeaways & Summary

Exam Tip



Always state your **n_0 threshold** clearly!

Inequalities like **$2n \leq 2n^2$** are only true when **$n \geq$**

Don't forget to specify **$n \geq n_0$**

Summary

Arithmetic Relation Asymptotic Notation Equivalent Meaning

$a \leq b$

$f(n) = O(g(n))$

f grows no faster than g

$a \geq b$

$f(n) = \Omega(g(n))$

f grows no slower than g

$a = b$

$f(n) = \Theta(g(n))$

f grows at the exact same rate

$a < b$

$f(n) = o(g(n))$

f grows strictly slower than g

$a > b$

$f(n) = \omega(g(n))$

f grows strictly faster than g

Mathematical Properties of Asymptotic Notation

Let $f(n)$, $g(n)$, and $h(n)$ be non-negative functions.

1. **Transitivity:**
2. **Reflexivity:**
3. **Symmetry:**
4. **Transpose Symmetry:**

Mathematical Properties of Asymptotic Notation

Let $f(n)$, $g(n)$, and $h(n)$ be non-negative functions.

1. **Transitivity:**

- $f(n) = O(g(n))$ AND $g(n) = O(h(n)) \Rightarrow f(n) = O(h(n))$
- $f(n) = \Theta(g(n))$ AND $g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$

2. **Reflexivity:**

- $f(n) = O(f(n))$, $f(n) = \Omega(f(n))$, $f(n) = \Theta(f(n))$

3. **Symmetry:**

- $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$
- $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$

4. **Transpose Symmetry:**

- $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$

Limit Rules for Asymptotic Comparison

Comparing the efficiency of two complex algorithms

L'Hôpital's rule applies to Big O algorithmic comparisons because Big O notation relies on limits at infinity to compare the growth rates of two functions, which often results in indeterminate forms like ∞ / ∞

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

Limit Rules for Asymptotic Comparison

Let $L = \lim_{n \rightarrow \infty} [f(n) / g(n)]$

If $L = 0$

- $f(n)$ grows strictly slower than $g(n)$.
- $f(n) = O(g(n))$

If $L = c$ (where c is a positive constant, $0 < c < \infty$)

- $f(n)$ and $g(n)$ grow at the same rate.
- $f(n) = \Theta(g(n))$, which implies $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Limit Rules for Asymptotic Comparison

Let $L = \lim_{n \rightarrow \infty} [f(n) / g(n)]$

If $L = \text{infinity}$

- $f(n)$ grows strictly faster than $g(n)$.
- $f(n) = \Omega(g(n))$

If L does not exist (oscillates)

- L'Hopital's rule is inconclusive.
- You must use alternative bounds methods to prove the relationship.

Asymptotic Growth Rate Hierarchy

Ordered from slowest growing (most efficient) to fastest growing (least efficient):

$\Theta(1) < \Theta(\log \log n) < \Theta(\log n) < \Theta(\sqrt{n}) < \Theta(n) < \Theta(n \log n) < \Theta(n^2) < \Theta(n^3) < \Theta(2^n) < \Theta(n!)$

- **Logarithmic vs. Polynomial:** Logarithms grow slower than *any* positive polynomial power.
- **Polynomial vs. Exponential:** Polynomials grow slower than *any* exponential base.

Practical Impact of Growth Rates

Execution time assuming **1 billion (10^9) primitive operations per second**:

- **$O(\log n)$:**
 - $n = 100$: 6.6 nanoseconds
 - $n = 10,000$: 13.3 nanoseconds
 - $n = 1,000,000$: 19.9 nanoseconds
- **$O(n)$:**
 - $n = 100$: 0.1 microseconds
 - $n = 10,000$: 10 microseconds
 - $n = 1,000,000$: 1 millisecond
- **$O(n \log n)$:**
 - $n = 100$: 0.66 microseconds
 - $n = 10,000$: 0.13 milliseconds
 - $n = 1,000,000$: 19.9 milliseconds

Execution time assuming **1 billion (10^9) primitive operations per second**:

- **$O(n^2)$:**
 - $n = 100$: 10 microseconds
 - $n = 10,000$: 0.1 seconds
 - $n = 1,000,000$: **11.5 days**
- **$O(n^3)$:**
 - $n = 100$: 1 millisecond
 - $n = 10,000$: **16.6 minutes**
 - $n = 1,000,000$: **31,710 years**
- **$O(2^n)$:**
 - $n = 100$: **4×10^{13} years**
 - $n = 10,000$: **End of Universe**
 - $n = 1,000,000$: **End of Universe**

Misconceptions

- ❑ Assuming Big-O always means "worst case."
- ❑ Dropping variables in multi-variable inputs.
- ❑ Believing $O(n^2)$ is always slower than $O(n)$.

Misconceptions

- ❑ Assuming Big-O always means "worst case."

Big-O is a mathematical ceiling. You can analyze the best case in Big-O, or the worst case in Big-Omega.

- ❑ Dropping variables in multi-variable inputs.

- ❑ Believing $O(n^2)$ is always slower than $O(n)$.

Misconceptions

- ❑ Assuming Big-O always means "worst case."

Big-O is a mathematical ceiling. You can analyze the best case in Big-O, or the worst case in Big-Omega.

- ❑ Dropping variables in multi-variable inputs.

Graph algorithms running on V vertices and E edges are $O(V + E)$, **not** $O(N)$.

- ❑ Believing $O(n^2)$ is always slower than $O(n)$.

Misconceptions

- ❑ Assuming Big-O always means "worst case."

Big-O is a mathematical ceiling. You can analyze the best case in Big-O, or the worst case in Big-Omega.

- ❑ Dropping variables in multi-variable inputs.

Graph algorithms running on V vertices and E edges are $O(V + E)$, **not** $O(N)$.

- ❑ Believing $O(n^2)$ is always slower than $O(n)$.

For small n ($n < n_0$), constant factors dominate. $1000n$ is slower than n^2 when $n < 1000$.

Summary & next lecture

Asymptotic notation provides a hardware-independent mathematical framework for measuring algorithm scalability.

- $O(g(n)) \Rightarrow$ Upper bound ($\leq$)
- $\Omega(g(n)) \Rightarrow$ Lower bound ($\geq$)
- $\Theta(g(n)) \Rightarrow$ Tight bound ($=$)

Next Lecture: Applying these definitions to analyze lines of code, loops, and conditional structures.